QuickSort: implementação e análise de complexidade

Pivô aleatório (caso médio) e partição proporcional α : (1 − α)

Sumário

- A. Introdução
  - O problema
  - Visão geral do programa
- B. Implementação
  - Estruturas de dados e tipos abstratos
  - Escolha do particionamento: Lomuto
  - Versão i: pivô aleatório
  - Versão ii: QuickSort Proporcional
  - Tratamento do `StackOverflowError`
  - Geração dos vetores
  - Entrada e saída
  - Resultados obtidos
- C. Análise de complexidade
  - i. Procedimento de particionamento (Lomuto)
  - ii. QuickSort com pivô aleatório (caso médio)
  - iii. QuickSort Proporcional
  - Quadro-resumo
- Referências

A. Introdução

O problema

Ordenar um arranjo de n inteiros é colocá-los em ordem crescente. O **QuickSort** resolve isso por *divisão e conquista*:

1. **Dividir:** escolhe-se um elemento, o **pivô**, e reorganiza-se o arranjo de modo que todos os elementos menores que o pivô fiquem à sua esquerda e todos os maiores fiquem à sua direita. Depois disso, o pivô já está na sua posição definitiva.
2. **Conquistar:** ordenam-se, recursivamente, a parte da esquerda e a parte da direita.
3. **Combinar:** nada a fazer. Como cada parte já está ordenada e no lugar certo, o arranjo inteiro fica ordenado.

Todo o trabalho "de verdade" está no passo 1, o **particionamento**. A eficiência do algoritmo depende de *quão equilibradas* são as duas partes geradas, e isso depende só da escolha do pivô. Por isso este trabalho implementa **um único procedimento de particionamento** e varia apenas a forma de escolher o pivô:

- **QuickSort com pivô aleatório:** o pivô é sorteado. Todas as divisões são igualmente prováveis, e a análise mostra que o tempo **esperado** é $\Theta(n \log n)$.
- **QuickSort Proporcional:** recebe um parâmetro α ($0 < \alpha \leq 1/2$) e escolhe o pivô de forma a **forçar** que todo subarranjo de tamanho n seja dividido em partes de tamanhos αn e $(1-\alpha)n$. Foram escolhidos $\alpha = 1/2$ (partição 50%–50%) e $\alpha = 1/4$ (partição 25%–75%) para a análise detalhada; o programa também executa $\alpha = 1/3$ e $\alpha = 1/5$.

As entradas são **permutações aleatórias dos inteiros de 1 até** n, sem valores repetidos.

Visão geral do programa

O programa (classe `Main`) executa dois experimentos:

1. **Experimento 1:** para $n \in \{10^3, 10^4, 10^5, 10^6\}$, gera 5 permutações aleatórias de cada tamanho e ordena cada uma com as cinco versões (pivô aleatório e proporcional com $\alpha = 1/2, 1/3, 1/4, 1/5$). Após cada ordenação, **confere se o resultado é exatamente** $1, 2, \ldots, n$. Mostra o número de comparações, o valor previsto pela análise, a altura do ramo mais curto ($h_{\min}$) e do mais longo ($h_{\max}$) da árvore de recursão, a profundidade real da pilha e o tempo.
2. **Experimento 2:** executa a versão em que a partição **não é proporcional** (sempre 0 e $n-1$), para mostrar o comportamento quadrático e que o programa não sofre `StackOverflowError`.

Os resultados também são gravados em `resultados.csv`.

Compilar e executar (Java 17 ou superior):

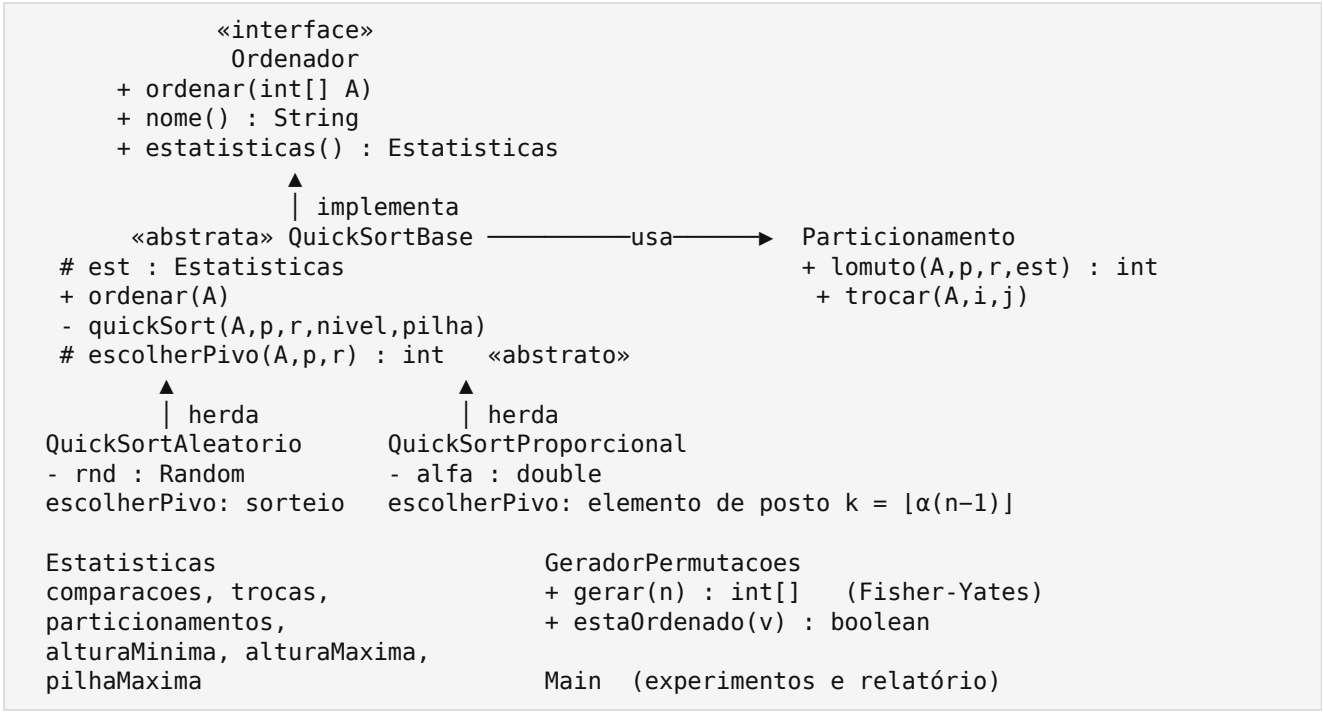
```
javac -d bin src/*.java
java -cp bin Main
```

Opções: `--n 1000,10000` (tamanhos), `--rep 5` (repetições), `--semente 42` (semente do gerador aleatório, para reproduzir os resultados), `--degenerado 1000,5000` (tamanhos do experimento 2) e `--csv arquivo.csv`.

B. Implementação

Estruturas de dados e tipos abstratos

Os dados são arranjos `int[]`, ordenados no próprio arranjo (*in place*), sem memória auxiliar além da pilha de recursão. Os tipos abstratos de dados foram organizados assim:



Classe	Papel
`Ordenador`	Interface: qualquer algoritmo de ordenação de `int[]`.
`QuickSortBase`	Esqueleto do QuickSort, comum às duas versões. Define o método abstrato `escolherPivo`.
`Particionamento`	O **único** procedimento de particionamento (Lomuto), usado por todas as versões.
`QuickSortAleatorio`	Versão i: sorteia o índice do pivô.
`QuickSortProporcional`	Versão ii: recebe α e escolhe o pivô que força a divisão $\alpha:(1-\alpha)$.
`Estatisticas`	Contadores coletados durante a ordenação.
`GeradorPermutacoes`	Gera as permutações aleatórias de $1..n$ e confere a ordenação.
`Main`	Roda os experimentos, compara com as previsões teóricas e grava o CSV.

Este desenho (método *template*) garante que as duas versões compartilham exatamente o mesmo código de particionamento e de recursão; a única diferença entre elas é uma linha: qual índice é devolvido por `escolherPivo`.

Escolha do particionamento: Lomuto

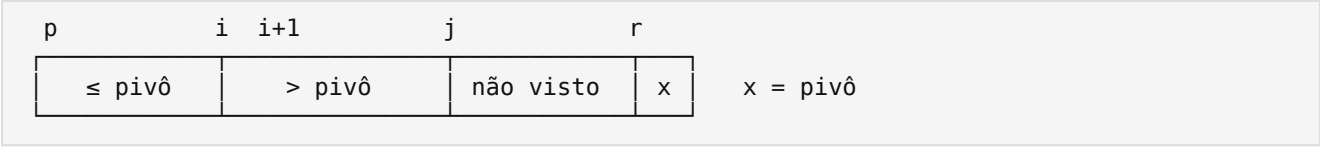
Foi usado o particionamento de **Lomuto**. Justificativa:

1. **O pivô termina na posição definitiva** q, e o subarranjo é dividido exatamente em `A[p..q-1]` (menores) e `A[q+1..r]` (maiores). Os tamanhos dos subproblemas são exatamente k e $n-1-k$, que é a forma das recorrências da Parte II.
2. Com isso, **forçar a proporção** é simples: basta escolher como pivô o elemento que deve ficar na posição $p+k$. No esquema de Hoare o pivô não termina

necessariamente na posição final, e o tamanho das partes depende da disposição dos dados, o que tornaria a versão proporcional muito mais difícil de controlar.
3. Faz **exatamente** $n-1$ **comparações** para n elementos, o que é fácil de contar e de analisar.
4. Sua fraqueza conhecida (muitos elementos iguais) não acontece aqui, porque as entradas não têm valores repetidos.

O pivô escolhido por qualquer uma das versões é primeiro trocado com `A[r]`, a posição onde o Lomuto o espera.

Funcionamento. O índice `i` marca o fim da região dos elementos $\leq$ pivô; o índice `j` percorre o restante. Quando `A[j]` é $\leq$ pivô, ele é trazido para a região da esquerda com uma troca. No final, o pivô é trocado com `A[i+1]`. Invariante do laço:



Exemplo com `A = [3, 8, 2, 5, 1, 4, 7, 6]` e pivô $x = 6$:

j	A[j]	≤ 6?	ação	arranjo
0	3	sim	`i=0`, troca(0,0)	`[3, 8, 2, 5, 1, 4, 7, 6]`
1	8	não	—	`[3, 8, 2, 5, 1, 4, 7, 6]`
2	2	sim	`i=1`, troca(1,2)	`[3, 2, 8, 5, 1, 4, 7, 6]`
3	5	sim	`i=2`, troca(2,3)	`[3, 2, 5, 8, 1, 4, 7, 6]`
4	1	sim	`i=3`, troca(3,4)	`[3, 2, 5, 1, 8, 4, 7, 6]`
5	4	sim	`i=4`, troca(4,5)	`[3, 2, 5, 1, 4, 8, 7, 6]`
6	7	não	—	`[3, 2, 5, 1, 4, 8, 7, 6]`
fim			troca(5,7)	`[3, 2, 5, 1, 4,`**`6`**`, 7, 8]`

Resultado: $q = 5$, partes de tamanho 5 e 2, com $7 = n - 1$ comparações.

Versão i: pivô aleatório

`escolherPivo` devolve `p + rnd.nextInt(r - p + 1)`, um índice sorteado de forma uniforme entre as $r - p + 1$ posições do subarranjo. Usa-se `java.util.Random` com semente fixa para os experimentos serem reproduzíveis.

Versão ii: QuickSort Proporcional

Para dividir um subarranjo de n elementos na razão $\alpha : (1-\alpha)$, a parte esquerda deve ter

$$k = \lfloor \alpha\,(n-1) \rfloor$$

elementos (os $n-1$ elementos que não são o pivô são repartidos), e a direita $n-1-k$. Para isso, o pivô deve ser o $(k+1)$**-ésimo menor elemento** de `A[p..r]`.

Como achá-lo. O QuickSort mantém uma propriedade importante: depois de cada particionamento, o subarranjo `A[p..r]` contém **exatamente** os elementos que ocuparão as posições $p..r$ no vetor ordenado. Como a entrada é uma permutação de $1..n$ (e os índices em Java começam em 0), o subarranjo `A[p..r]` contém exatamente os valores $\{p+1, \ldots, r+1\}$. Logo, o pivô procurado é o **valor** $p+1+k$, e basta uma busca linear para achar sua posição.

Essa busca custa $\Theta(n)$, o mesmo que o próprio particionamento. Portanto ela só aumenta a constante c do termo cn da recorrência; a complexidade assintótica não muda. (Para entradas quaisquer, o mesmo efeito seria obtido com um algoritmo de seleção em tempo linear, como o da *mediana das medianas*, também $\Theta(n)$. Como o enunciado garante permutações de $1..n$, foi usada a busca, que é mais simples.) Se o valor não for encontrado, o programa lança uma exceção, avisando que a entrada não é uma permutação.

Com $a = 0$ tem-se $k = 0$: o pivô é sempre o **menor** elemento e a partição é sempre 0 e $n-1$. Esse valor é usado só no Experimento 2, para demonstrar o caso não proporcional.

Tratamento do `StackOverflowError`

Na versão “de livro”, o QuickSort faz duas chamadas recursivas. Quando as partições são ruins (por exemplo, 0 e $n-1$), a recursão chega a profundidade $n-1$; com n na casa das dezenas de milhares a pilha da JVM estoura e ocorre `StackOverflowError`.

Decisão tomada: fazer a chamada recursiva **apenas para a parte menor** e tratar a **parte maior por iteração**, com um laço `while` que reaproveita a mesma chamada (eliminação da recursão de cauda):

```
        while (p < r) {
            int iPivo = escolherPivo(A, p, r);
            Particionamento.trocar(A, iPivo, r);
            int q = Particionamento.lomuto(A, p, r, est);
            if (q - p < r - q) {            // esquerda é a menor
                quickSort(A, p, q - 1, ...); // recursão na menor
                p = q + 1;                   // laço continua na maior
            } else {                         // direita é a menor
                quickSort(A, q + 1, r, ...);
                r = q - 1;
            }
        }
```

Por que funciona: a parte menor tem **no máximo metade** dos elementos. Assim, cada nova chamada empilhada recebe no máximo metade do tamanho da anterior, e a profundidade da pilha fica limitada a $\log_2 n$ (cerca de 20 para $n = 10^6$), **em qualquer caso, inclusive no pior**. O número de comparações e o tempo não mudam; muda apenas o uso de memória da pilha, de $O(n)$ para $O(\log n)$. Não foi preciso aumentar o tamanho da pilha da JVM (`-Xss`).

```
 Versão ingênua (partição 0 / n-1)     Versão usada
 pilha:  quickSort(n)                   pilha: quickSort(n)  ── laço: n, n-1, n-2, ... 1
          quickSort(n-1)                          └ quickSort(0)  (parte menor, retorna
logo)
           quickSort(n-2)
            ...           ← n níveis    profundidade máxima: 2
```

No Experimento 2 o programa mede isso: com $n = 20\,000$ e partições $0 / n-1$, a árvore de recursão tem altura $19\,998$, mas a pilha real nunca passa de 2 chamadas.

O programa distingue, portanto, duas medidas:

- **altura da árvore de recursão** ($h_{\min}$ e $h_{\max}$): a estrutura lógica das subdivisões, a mesma da análise teórica. Cada subarranjo filho conta como um nível a mais, seja ele tratado por recursão ou por iteração;
- **profundidade da pilha**: quantas chamadas estão de fato empilhadas.

Geração dos vetores

`GeradorPermutacoes.gerar(n)` preenche o vetor com $1, 2, \ldots, n$ e o embaralha com o algoritmo de **Fisher–Yates**: para i de $n-1$ até 1, troca a posição i com uma posição sorteada em $[0, i]$. Isso produz cada uma das $n!$ permutações com a mesma probabilidade, em tempo $\Theta(n)$, e garante que não há valores repetidos. A semente é configurável, de modo que os mesmos vetores podem ser gerados de novo.

Entrada e saída

- **Entrada:** argumentos opcionais na linha de comando (tamanhos, repetições, semente). Os vetores são gerados pelo próprio programa.
- **Saída:** uma tabela no console para cada versão e o arquivo `resultados.csv`. As colunas são: n; número médio de comparações; valor previsto pela análise; razão comparações $/(n \log_2 n)$; constante assintótica teórica; $h_{\min}$ e $h_{\max}$ medidos e previstos; profundidade máxima da pilha; e tempo médio em milissegundos.
- Se alguma ordenação produzir um resultado errado, o programa para com uma exceção.

Resultados obtidos

Saída do programa (semente 42, 5 repetições; os tempos variam conforme a máquina):

```
QuickSort pivo aleatorio
         n      comparacoes         previsao   C/nlog2n  assint.   hmin   prev   hmax
prev  pilha
      1000            11110            10986      1.115    1.386      3            22
8
     10000           152811           155772      1.150    1.386      4            32
10
    100000          2023248          2018053      1.218    1.386      4            41
13
   1000000         25022134         24785482      1.255    1.386      6            51
14

QuickSort proporcional alfa=1/2
      1000             7987             7987      0.801    1.000      9   10.0      8
10.0     10
     10000           113631           113631      0.855    1.000     12   13.3     12
13.3     13
    100000          1468946          1468946      0.884    1.000     16   16.6     15
16.6     17
   1000000         17951445         17951445      0.901    1.000     19   19.9     18
19.9     20

QuickSort proporcional alfa=1/4
      1000            10001            10001      1.004    1.233      5    5.0     20
24.0      6
     10000           141583           141583      1.066    1.233      7    6.6     28
32.0      8
    100000          1824864          1824864      1.099    1.233      8    8.3     36
40.0      9
   1000000         22303483         22303483      1.119    1.233     10   10.0     44
48.0     11

QuickSort proporcional alfa=0 (degenerado, partição 0 e n-1)
         n      comparacoes         n(n-1)/2                     hmin          hmax
pilha
      1000           499500           499500                        1           998
2
      5000         12497500         12497500                        1          4998
2
     20000        199990000        199990000                        1         19998
2
```

Observações:

- **Pivô aleatório:** a média medida fica muito próxima do valor esperado exato $2(n+1)H_n - 4n$ deduzido na Parte II (diferença abaixo de 2%).
- **Proporcional:** o número de comparações é **idêntico** ao valor obtido resolvendo numericamente a recorrência $C(n) = (n-1) + C(k) + C(n-1-k)$, o que confirma que a implementação força exatamente a divisão pretendida. As alturas medidas acompanham $\log_{1/\alpha} n$ e $\log_{1/(1-\alpha)} n$ (ficam um pouco abaixo por causa dos arredondamentos e porque a recursão para em tamanho 1).
- A razão $C/(n \log_2 n)$ cresce devagar em direção à constante assintótica, porque termos de ordem n (negativos) ainda pesam para os valores de n usados.
- **Não proporcional:** exatamente $n(n-1)/2$ comparações, ou seja, crescimento quadrático: multiplicar n por 4 (de 5 000 para 20 000) multiplicou as comparações por 16.

C. Análise de complexidade

Notação: $T(n)$ é o tempo para ordenar um subarranjo de n elementos, e c é uma constante positiva. Os logaritmos sem base indicada podem ser tomados em qualquer base, pois diferem só por uma constante.

i. Procedimento de particionamento (Lomuto)

Para um subarranjo `A[p..r]` com $n = r - p + 1$ elementos:

- antes do laço há duas atribuições (`pivo = A[r]`, `i = p-1`): custo constante a;
- o laço `for` executa para $j = p, \ldots, r-1$, isto é, **exatamente** $n-1$ **vezes**, para qualquer entrada. Em cada volta há **uma** comparação `A[j] <= pivo` e, no máximo, um incremento e uma troca: custo de cada volta entre b_1 e b_2, constantes;
- depois do laço há uma troca e o retorno: custo constante d.

Então o custo total satisfaz

$$a + d + b_1\,(n-1) \;\le\; P(n) \;\le\; a + d + b_2\,(n-1).$$

O lado direito mostra que $P(n) = O(n)$; o lado esquerdo mostra que $P(n) = \Omega(n)$, porque o laço é percorrido inteiro **sempre**, independentemente de quais elementos são menores ou maiores que o pivô (não existe “saída antecipada”). Logo

$$P(n) = \Theta(n).$$

A escolha do pivô também é no máximo linear: $\Theta(1)$ no aleatório e $\Theta(n)$ (busca linear) no proporcional. Somando tudo o que o QuickSort faz em uma chamada antes de recursionar, o custo é $\Theta(n)$, ou seja, no máximo e no mínimo proporcional a n. Esse é o termo cn que aparece em todas as recorrências a seguir.

ii. QuickSort com pivô aleatório (caso médio)

Recorrência

O pivô é sorteado uniformemente entre os n elementos. Como os valores são distintos, cada elemento tem um **posto** (posição no vetor ordenado) diferente. Se o pivô sorteado tem posto $k+1$, a parte esquerda fica com k elementos e a direita com $n-1-k$. Portanto o tamanho k da parte esquerda vale $0, 1, \ldots, n-1$, cada um com probabilidade $1/n$. Tomando a média sobre essas n possibilidades:

$$T(n) = \frac{1}{n}\sum_{k=0}^{n-1}\left[T(k) + T(n-1-k)\right] + cn, \qquad T(0) = T(1) = 0 \text{ (ou constante).}$$

Juntando os dois somatórios em um só

Separando a soma em duas:

$$\sum_{k=0}^{n-1} T(k) = T(0) + T(1) + \cdots + T(n-1)$$

$$\sum_{k=0}^{n-1} T(n-1-k) = T(n-1) + T(n-2) + \cdots + T(0)$$

São **os mesmos termos**, só que o segundo está em ordem inversa (quando k vai de 0 a $n-1$, $n-1-k$ vai de $n-1$ a 0). Logo as duas somas são iguais, e

$$T(n) = \frac{2}{n}\sum_{k=0}^{n-1} T(k) + cn. \qquad (1)$$

Intuitivamente: cada tamanho j aparece duas vezes, uma vez como "parte esquerda" e outra como "parte direita".

Eliminando o somatório

Multiplicando (1) por n:

$$n\,T(n) = 2\sum_{k=0}^{n-1} T(k) + cn^2. \qquad (2)$$

A mesma equação, escrita para $n-1$:

$$(n-1)\,T(n-1) = 2\sum_{k=0}^{n-2} T(k) + c(n-1)^2. \qquad (3)$$

Subtraindo (3) de (2), quase todo o somatório se cancela; sobra apenas o termo $k = n-1$:

$$n\,T(n) - (n-1)\,T(n-1) = 2\,T(n-1) + c(n^2 - (n-1)^2) = 2\,T(n-1) + c(2n-1).$$

Passando $(n-1)T(n-1)$ para a direita:

$$n\,T(n) = (n+1)\,T(n-1) + c(2n-1).$$

Agora divide-se tudo por $n(n+1)$, para que o lado esquerdo e o primeiro termo da direita fiquem com a "mesma cara":

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{c(2n-1)}{n(n+1)}. \qquad (4)$$

Telescopando até $T(1)$

Chamando $S(n) = \dfrac{T(n)}{n+1}$, a equação (4) diz que $S(n) = S(n-1) + c\,\dfrac{2n-1}{n(n+1)}$. Aplicando-a repetidamente:

$$\begin{aligned} S(n) &= S(n-1) + c\,\frac{2n-1}{n(n+1)} \\ &= S(n-2) + c\,\frac{2(n-1)-1}{(n-1)n} + c\,\frac{2n-1}{n(n+1)} \\ &\;\;\vdots \\ &= S(1) + c\sum_{i=2}^{n} \frac{2i-1}{i(i+1)}. \end{aligned}$$

Cada termo cancela o anterior, como as partes de uma luneta que se fecha (daí o nome *telescopar*): sobram só o primeiro termo, $S(1)$, e a soma das parcelas.

Relação com a série harmônica

Decompondo a fração em frações parciais:

$$\frac{2i-1}{i(i+1)} = \frac{3}{i+1} - \frac{1}{i}$$

(confere: $\frac{3i-(i+1)}{i(i+1)} = \frac{2i-1}{i(i+1)}$). Usando a **série harmônica** $H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$:

$$\sum_{i=2}^{n} \frac{3}{i+1} = 3(H_{n+1} - 1 - \tfrac{1}{2}), \qquad \sum_{i=2}^{n} \frac{1}{i} = H_n - 1.$$

Como $H_{n+1} = H_n + \frac{1}{n+1}$:

$$\sum_{i=2}^{n} \frac{2i-1}{i(i+1)} = 2H_n + \frac{3}{n+1} - \frac{7}{2}.$$

Voltando para $T(n) = (n+1)S(n)$:

$$T(n) = (n+1)[\,S(1) + c(2H_n + \frac{3}{n+1} - \frac{7}{2})] = 2c\,(n+1)H_n + O(n).$$

A série harmônica cresce como o logaritmo natural: $H_n = \ln n + \gamma + O(1/n)$, com $\gamma \approx 0{,}577$ (constante de Euler–Mascheroni). Isso pode ser visto comparando a soma com a área sob $1/x$: $\ln(n+1) \le H_n \le 1 + \ln n$. Portanto

$$T(n) = 2c\,n \ln n + O(n) \approx 1{,}39\,c\,n \log_2 n,$$

$$T(n) = \Theta(n \log n) \text{ (tempo esperado)}$$

Conferência com o número exato de comparações. Se em vez de cn usamos o custo exato do Lomuto, $n-1$ comparações, a mesma dedução (com $\frac{2(i-1)}{i(i+1)} = \frac{4}{i+1} - \frac{2}{i}$ e $C(1) = 0$) dá

$$E[C(n)] = 2(n+1)H_n - 4n.$$

É esse o valor da coluna “previsão” do programa. Para $n = 10^6$ dá $24\,785\,482$, e a média medida foi $25\,022\,134$ (diferença de cerca de 1%).

Por que vale para qualquer sequência de entrada

Na análise acima, a aleatoriedade **não está nos dados**, mas nas escolhas do algoritmo. Para qualquer entrada fixa (ordenada, invertida, a pior imaginável), o pivô sorteado em um subarranjo de tamanho m é o elemento de posto j com probabilidade $1/m$, para cada j, porque o sorteio é uniforme sobre as posições e os valores são distintos. Ou seja, a distribuição dos tamanhos das partições é **sempre** uniforme, e a recorrência (1) vale igualmente para todas as entradas. Nenhuma entrada específica consegue “enganar” o algoritmo de forma sistemática: o tempo **esperado** é $\Theta(n \log n)$ para toda entrada. (Diferente do QuickSort com pivô fixo, por exemplo o último elemento, em que uma entrada já ordenada provoca sempre o pior caso.)

O pior caso ainda pode ocorrer

Sim. Nada impede que, por azar, o sorteio escolha sempre o menor (ou o maior) elemento do subarranjo. Aí as partições são sempre 0 e $n-1$ e o tempo é $\Theta(n^2)$ (ver o final do item iii). A diferença é que isso deixa de depender da entrada e passa a depender só

da sorte, e a probabilidade é minúscula: em cada subarranjo de tamanho m a chance de escolher um extremo é $2/m$, e precisa acontecer em quase todos os níveis, o que dá probabilidade $\prod_{m=2}^{n} \frac{2}{m} = \frac{2^{n-1}}{n!}$, desprezível já para n pequeno ($\approx 1{,}4 \times 10^{-4}$ para $n = 10$). Em resumo: **pior caso** $\Theta(n^2)$**, caso esperado** $\Theta(n \log n)$**, melhor caso** $\Theta(n \log n)$**.**

iii. QuickSort Proporcional

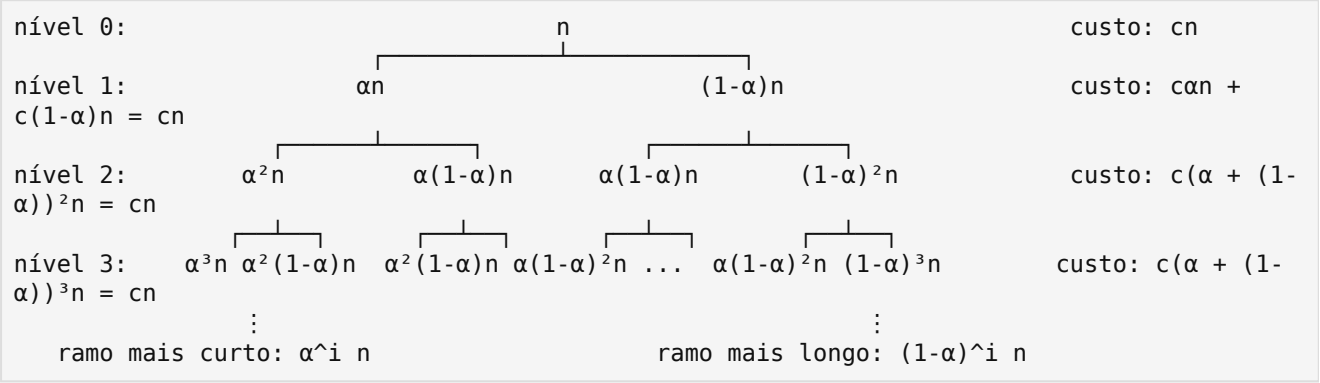
Supõe-se que o pivô sempre divide o subarranjo de tamanho n em partes de tamanho αn e $(1-\alpha)n$, com $0 < \alpha \leq 1/2$ constante (desprezando arredondamentos e o próprio pivô, que não alteram a ordem de grandeza):

$$T(n) = T(\alpha n) + T((1-\alpha)n) + cn.$$

Primeiro faz-se a análise para um α genérico, e depois aplica-se aos valores escolhidos, $\alpha = 1/2$ e $\alpha = 1/4$.

Árvore de recursão (caso geral)

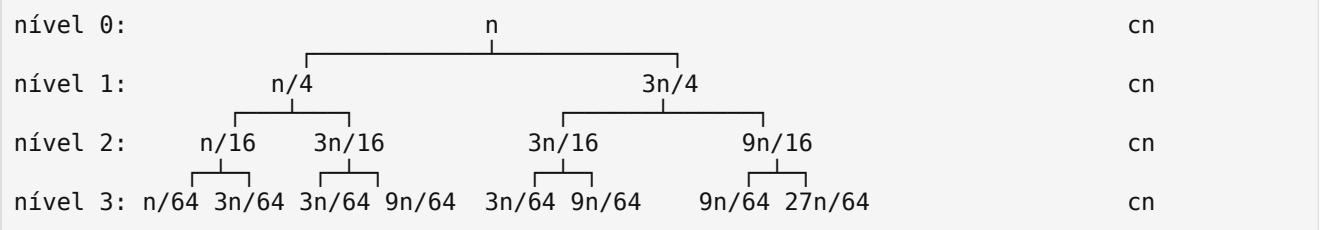
Cada nó mostra o tamanho do subproblema; seu custo é c vezes esse tamanho.



No nível 3 há um nó de tamanho $\alpha^3 n$, três de tamanho $\alpha^2(1-\alpha)n$, três de tamanho $\alpha(1-\alpha)^2 n$ e um de tamanho $(1-\alpha)^3 n$. Esses são os termos do binômio $(\alpha + (1-\alpha))^3$, cuja soma é $1^3 = 1$. O mesmo vale para todo nível i: a soma dos tamanhos é $(\alpha + (1-\alpha))^i n = n$. Isso faz sentido: cada nível apenas reparte os mesmos elementos (menos os pivôs) em pedaços menores. Portanto:

- **todo nível completo custa exatamente** cn;
- os níveis depois que o ramo mais curto terminou estão incompletos e custam **no máximo** cn.

Exemplo com $\alpha = 1/4$**:**



(Conferindo o nível 3: $\frac{1+3+3+9+3+9+9+27}{64} = \frac{64}{64} = 1$.)

Exemplo com $\alpha = 1/2$: todos os nós de um nível têm o mesmo tamanho: n; $n/2, n/2$; quatro vezes $n/4$; oito vezes $n/8$; e assim por diante. Cada nível custa cn e a árvore é perfeitamente equilibrada.

Altura do ramo mais curto e do mais longo

O **ramo mais curto** segue sempre a parte menor: no nível i o tamanho é $\alpha^i n$. Ele termina quando chega a 1:

$$\alpha^{h_{\min}} n = 1 \;\Rightarrow\; \left(\tfrac{1}{\alpha}\right)^{h_{\min}} = n \;\Rightarrow\; h_{\min} = \log_{1/\alpha} n.$$

O **ramo mais longo** segue sempre a parte maior: tamanho $(1-\alpha)^i n$ no nível i:

$$(1-\alpha)^{h_{\max}} n = 1 \;\Rightarrow\; h_{\max} = \log_{1/(1-\alpha)} n.$$

Como $\alpha \le 1-\alpha$, $h_{\min} \le h_{\max}$. Todas as folhas da árvore estão entre esses dois níveis.

Limite inferior e limite superior

Inferior. Do nível 0 até o nível $h_{\min}$ (exclusive) todos os níveis estão completos (nenhum ramo terminou ainda), e cada um custa cn. Ignorando o resto da árvore, que só aumentaria o custo:

$$T(n) \ge cn \cdot h_{\min} = cn \log_{1/\alpha} n = \frac{c}{\log_2 (1/\alpha)}\, n \log_2 n \;\Rightarrow\; T(n) = \Omega(n \log n).$$

Superior. A árvore tem no máximo $h_{\max} + 1$ níveis (do 0 até o $h_{\max}$), e cada nível custa no máximo cn:

$$T(n) \le cn\,(h_{\max} + 1) = cn \log_{1/(1-\alpha)} n + cn \;\Rightarrow\; T(n) = O(n \log n).$$

$h_{\max}$ na forma $k \cdot \log_2 n$

Pela mudança de base, $\log_b n = \dfrac{\log_2 n}{\log_2 b}$. Então

$$h_{\max} = \log_{1/(1-\alpha)} n = \frac{1}{\log_2 \frac{1}{1-\alpha}} \log_2 n = k \cdot \log_2 n, \qquad k = \frac{1}{\log_2 \frac{1}{1-\alpha}} = \frac{-1}{\log_2 (1-\alpha)}.$$

Analogamente, $h_{\min} = \dfrac{1}{\log_2 (1/\alpha)} \log_2 n$.

α	**partição**	$h_{\min}$	$h_{\max} = k \log_2 n$	**constante** k
1/2	50%–50%	$\log_2 n$	$\log_2 n$	$\frac{1}{\log_2 2} = \mathbf{1}$
1/3	33%–67%	$\approx 0{,}631 \log_2 n$	$\log_{3/2} n$	$\frac{1}{\log_2 1{,}5} \approx 1{,}710$
1/4	25%–75%	$\frac{1}{2} \log_2 n$	$\log_{4/3} n$	$\frac{1}{\log_2 (4/3)} = \frac{1}{2 - \log_2 3} \approx \mathbf{2{,}409}$
1/5	20%–80%	$\approx 0{,}431 \log_2 n$	$\log_{5/4} n$	$\frac{1}{\log_2 1{,}25} \approx 3{,}106$

Para $n = 10^6$ ($\log_2 n \approx 19{,}93$): com $\alpha = 1/4$, $h_{\min} \approx 10{,}0$ e $h_{\max} \approx 48{,}0$. O programa mediu 10 e 44.

Aplicando aos valores escolhidos

$\alpha = 1/2$**:** $h_{\min} = h_{\max} = \log_2 n$ e $k = 1$. Todos os níveis estão completos, então os limites coincidem:

$$T(n) = cn \log_2 n + O(n) = \Theta(n \log n).$$

Esse é o melhor caso possível do QuickSort.

$\alpha = 1/4$**:** $\frac{c}{2} n \log_2 n \le T(n) \le 2{,}41\, c\, n \log_2 n + cn$, logo

$$T(n) = \Theta(n \log n),$$

com constante maior que a de $\alpha = 1/2$: a árvore é “torta” e mais alta do lado da parte maior.

Conclusão

Para **qualquer** α constante com $0 < \alpha \le 1/2$, os limites inferior e superior são ambos da forma (constante) $\times\, n \log n$, então

$$T(n) = \Theta(n \log n).$$

O valor de α só muda a **constante**. Quanto menor α, maior k e mais alta a árvore: com $\alpha = 1/100$ (partição 1%–99%), por exemplo, $k \approx 69$, e mesmo assim o algoritmo continua $\Theta(n \log n)$. É por isso que o QuickSort é rápido “quase sempre”: basta que as partições sejam proporcionais, não é preciso que sejam equilibradas. (Uma conta mais fina dá $T(n) \approx \frac{cn \log_2 n}{H(\alpha)}$, em que $H(\alpha) = -\alpha \log_2 \alpha - (1-\alpha) \log_2 (1-\alpha)$; esse valor fica entre os dois limites acima e corresponde à coluna “assint.” do programa: 1,000 para $\alpha = 1/2$ e 1,233 para $\alpha = 1/4$.)

E se a partição não for proporcional?

Suponha que a partição seja sempre 0 e $n-1$ (o pivô é sempre o menor ou o maior elemento). A parte menor não é mais uma **fração** de n: ela tem tamanho constante. A recorrência passa a ser

$$T(n) = T(n-1) + T(0) + cn = T(n-1) + cn.$$

Expandindo:

$$T(n) = cn + c(n-1) + c(n-2) + \cdots + c \cdot 2 + T(1) = c \sum_{i=2}^{n} i + T(1) = c\left(\frac{n(n+1)}{2} - 1\right) + T(1) = \Theta(n^2).$$

A árvore de recursão degenera num **caminho** de altura $n-1$ (cada nível tira só um elemento, o pivô), e os custos dos níveis formam uma progressão aritmética $cn, c(n-1), \ldots$ em vez de ficarem todos iguais a cn. O que muda em relação ao caso proporcional:

- a **altura** passa de $\Theta(\log n)$ para $\Theta(n)$, porque o tamanho diminui de **1** por nível (subtração), e não por um **fator** constante (divisão);
- o tempo passa de $\Theta(n \log n)$ para $\Theta(n^2)$;

- sem a otimização da seção B, a pilha teria profundidade n e haveria `StackOverflowError`.

O mesmo vale para qualquer partição em que a parte menor tem tamanho constante (por exemplo, 1 e $n-2$, ou 10 e $n-11$): a altura continua proporcional a n e o tempo continua $\Theta(n^2)$. O programa confirma isso: com $\alpha = 0$ fez exatamente $n(n-1)/2$ comparações ($199\,990\,000$ para $n = 20\,000$, contra cerca de $3{,}4 \times 10^5$ do pivô aleatório no mesmo tamanho).

Quadro-resumo

Versão / procedimento	Recorrência	Melhor caso	Caso médio / esperado	Pior caso	Pilha (com a otimização)
Particionamento de Lomuto	—	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$O(1)$
QuickSort, pivô aleatório	$T(n) = \frac{2}{n}\sum_{k=0}^{n-1} T(k) + cn$	$\Theta(n \log n)$	$\Theta(n \log n)$, $\approx 1{,}39\, n \log_2 n$ comparações	$\Theta(n^2)$ (prob. $\approx 2^{n-1}/n!$)	$O(\log n)$
Proporcional, $\alpha = 1/2$	$T(n) = 2T(n/2) + cn$	$\Theta(n \log n)$	$\Theta(n \log n)$, $h = \log_2 n$ ($k = 1$)	$\Theta(n \log n)$	$O(\log n)$
Proporcional, $\alpha = 1/4$	$T(n) = T(n/4) + T(3n/4) + cn$	$\Theta(n \log n)$	$\Theta(n \log n)$, $h_{\max} \approx 2{,}41 \log_2 n$	$\Theta(n \log n)$	$O(\log n)$
Proporcional, α constante qualquer em $(0, 1/2]$	$T(n) = T(\alpha n) + T((1-\alpha)n) + cn$	$\Theta(n \log n)$	$\Theta(n \log n)$, $h_{\max} = \frac{-1}{\log_2(1-\alpha)} \log_2 n$	$\Theta(n \log n)$	$O(\log n)$
Partição não proporcional (0 e $n-1$)	$T(n) = T(n-1) + cn$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$ (sem a otimização: $\Theta(n)$)

Na versão proporcional a divisão é imposta pelo próprio algoritmo, então melhor, médio e pior caso coincidem para um α fixo.

Referências

- CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Algoritmos: teoria e prática*. 3. ed. Rio de Janeiro: Elsevier, 2012. Capítulo 7 (Quicksort) e Capítulo 4 (método da árvore de recursão).
- ZIVIANI, N. *Projeto de Algoritmos com implementações em Java e C++*. São Paulo: Thomson, 2007.